

【试卷+答案】DS 强化结课考试

试卷结构：算法题 3 道 + 应用题 3 道，考试时长 120 分钟。

在 408 正式考试中，算法题通常约 15 分钟/题，应用题约 10 分钟/题。本次考试可适当放宽：算法题建议 25 分钟/题，应用题建议 15 分钟/题。若有知识点遗忘，可查阅王道教材（开卷）。

算法题

一、（20分）设非空线性表 L 采用带头结点的单链表存储，结点类型定义如下：

```
typedef struct LNode {
    int data;
    struct LNode *next;
} LNode, *LinkList;
```

若结点 a_i 的后方存在结点 a_j ，且 a_j 的值大于 a_i 的值，则称结点 a_i 为**受支配结点**。

例如，若 $L = (12, 15, 10, 11, 5, 6, 2, 3)$ ，则结点 12、10、5 和 2 均为受支配结点。删除这些结点后得到 $L' = (15, 11, 6, 3)$ 。

请设计一个时间上尽可能高效且额外空间复杂度为 $O(1)$ 的算法 `void deleteDominated(LinkList L)`，删除单链表中的所有受支配结点。要求不得申请新的链表结点，删除结点后其余结点的相对次序保持不变；若多个结点值相等，则它们之间不存在支配关系。要求如下：

- 1) 给出算法的基本设计思想。（6分）
- 2) 根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。（11分）
- 3) 说明所设计算法的时间复杂度和空间复杂度。（4分）

▼ 参考答案

1) 算法设计思想

1. 原地逆置整个单链表，使原链表中结点的右侧部分变为逆置链表中已经扫描过的部分。
2. 从前向后扫描逆置后的链表，用 `maxValue` 记录已扫描结点的最大值。
3. 若当前结点值小于 `maxValue`，说明它在原链表右侧存在更大的结点，将其删除。
4. 若当前结点值大于或等于 `maxValue`，则保留该结点，并更新 `maxValue`。
5. 再次逆置链表，恢复保留结点原来的相对次序。

注意易错点：由于题目中的支配关系要求“严格大于”，因此当前结点值等于 `maxValue` 时不能删除。

2) 算法实现

```
// 原地逆置带头结点的单链表
void reverseList(LinkList L) {
    LNode *p = L->next;
    LNode *q;
    L->next = NULL;
    while (p != NULL) {
        q = p->next;      // 保存尚未逆置的部分
        p->next = L->next;
        L->next = p;      // 将 p 插入头结点之后
        p = q;
    }
}

void deleteDominated(LinkList L) {
    LNode *pre, *p;
    int maxValue;
    // 空表或仅含一个数据结点时，不存在受支配结点
    if (L == NULL || L->next == NULL ||
        L->next->next == NULL)
        return;

    reverseList(L); // 单链表原地逆置

    pre = L->next;    // 前后双指针遍历、处理单链表
    p = pre->next;
    maxValue = pre->data;

    while (p != NULL) {
        if (p->data < maxValue) { // p 在原链表中是受支配节点，删除 p
            pre->next = p->next;
            free(p);
            p = pre->next;
        } else { // p->data ≥ maxValue, 不受支配，保留
            maxValue = p->data;
            pre = p;
            p = p->next;
        }
    }
}
```

```

    }
    reverseList(L);    //再次原地逆置，恢复原序
}

```

3) 时间、空间复杂度

设单链表中有 n 个数据结点。

- 第一次逆置、删除扫描和第二次逆置的时间复杂度均为 $O(n)$ ，因此总的时间复杂度为 $O(n)$ 。
- 算法空间复杂度为 $O(1)$ 。

▼ 评分细则与自助批改 (20 分)

档位	典型解法	时间复杂度	空间复杂度	总分上限
A	两次逆置，扫描时维护最大值	$O(n)$	$O(1)$	20 分
B	对每个结点重新扫描其右侧部分	$O(n^2)$	$O(1)$	16 分
C	能表达部分处理思路，但不能正确得到最终链表	—	—	不超过 8 分

二、(20分) 已知一棵非空树 T 采用孩子兄弟表示法存储，结点类型定义如下：

```

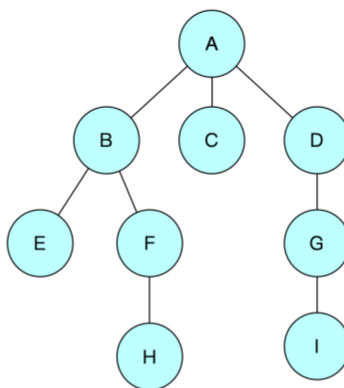
typedef struct CSNode {
    char data;           // 结点值
    struct CSNode *firstchild; // 指向最左边的孩子
    struct CSNode *nextsibling; // 指向右边的下一个兄弟
} CSNode, *CSTree;

```

其中，`firstchild` 指向结点的最左边孩子，`nextsibling` 指向该结点右边的下一个兄弟，树根的 `nextsibling` 为 `NULL`。

树中任意两个结点之间的距离定义为连接这两个结点的简单路径所包含的边数。树中任意两个结点距离的最大值称为树的直径。若树中只有一个结点，则其直径为 0。

例如，下图所示树的直径为 6，对应的一条最长路径为 $H \rightarrow F \rightarrow B \rightarrow A \rightarrow D \rightarrow G \rightarrow I$ 。



请设计一个时间上尽可能高效的算法 `int treeDiameter(CSTree T)`，在不改变树中结点及其链接关系的条件下，返回树 T 的直径。要求如下。

- 1) 给出算法的基本设计思想。(6分)
- 2) 根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。(11分)
- 3) 说明所设计算法的时间复杂度和空间复杂度。(4分)

▼ 参考答案

1) 算法设计思想

对树进行后根遍历。对每个结点 p ，沿孩子链递归计算各孩子的高度，维护从 p 向下的最大和次大分支长度 $max1$ 、 $max2$ ，用 $max1+max2$ 更新树的直径，并向双亲返回 $max1$ 。遍历结束后得到整棵树的直径。

2) 算法实现

```
int diameter = 0;    // 全局变量：记录树的直径。

/*
 * 返回结点 p 的高度，遍历过程中顺便更新全局直径 diameter。
 * 高度和直径均按路径中的边数计算。
 */
int getHeight(CSTree p)
{
    int max1 = 0;      // 从 p 出发的最长向下分支（对应最高子树的最远叶子）
    int max2 = 0;      // 从 p 出发的次长向下分支（对应次高子树的最远叶子）
    CSTree child;

    // 沿兄弟链依次处理 p 的所有孩子
    for (child = p->firstchild;
         child != NULL;
         child = child->nextsibling) {

        // branch 表示 p 到达子树 child 最远叶子的距离
        int branch = getHeight(child) + 1;

        // 维护最大的两个孩子分支
        if (branch > max1) {
            max2 = max1;
            max1 = branch;
        } else if (branch > max2) {
            max2 = branch;
        }
    }

    // 以 p 为根的路径是目前发现最长的一条，更新树的直径。
    if (max1 + max2 > diameter)
        diameter = max1 + max2;

    // 向双亲返回从 p 向下的一条最长分支
    return max1;
}

int treeDiameter(CSTree T) {
    diameter = 0;    // 每次调用前重置全局变量（直径），多次调用才不会出错
    if (T != NULL)
        getHeight(T);
}
```

```

return diameter;
}

```

3) 复杂度

设树中有 n 个结点，树高为 h 。每个结点只被递归处理一次，因此时间复杂度为 $O(n)$ ；递归栈深度最多为树高 h ，因此额外空间复杂度为 $O(h)$ 。

三、(20分) 中国高铁以“安全、快捷、舒适”的优势享誉世界，是中国自主创新的重大成就之一。如今，中国已建成全球最大规模的高速铁路网，实现的诸多城市之间的铁路贯通，向世界展示了“中国速度”和“中国智造”的实力。假设用一个无向图邻接矩阵表示 N 个城市（城市编号为 $0 \sim N-1$ ）之间的“直达列车”情况。例如，城市0、城市1之间有直达列车，则两个城市之间的无向边记为1；城市0、城市3之间没有直达列车，则两个城市之间的无向边记为0。如果两个城市之间没有直达列车，就需要换乘。请实现一个算法，计算从城市 `start` 到城市 `end`，最少需要乘坐几趟列车？

城市编号	0	1	2	3	4	5
0	0	1	1	0	0	0
1	1	0	0	1	1	0
2	1	0	0	0	1	1
3	0	1	0	0	0	1
4	0	1	1	0	0	1
5	0	0	1	1	1	0

- (1) 给出邻接矩阵的数据结构定义。(4分)
- (2) 给出算法的基本设计思想。(5分)
- (3) 根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。(11分)

【考后训练】将本题改为“邻接表存储”，再次实现算法。408 历年真题中，考察图的算法题截至目前都是基于“邻接矩阵存储”，接下来若再考图的算法，很可能基于“邻接表存储”。

▼ 参考答案

(1) 邻接矩阵定义

```
#define MAXV 100

typedef struct {
    int numVertices;           // 城市数，即顶点数
    int numEdges;              // 直达线路数，即无向边数
    int Edge[MAXV][MAXV];      // 邻接矩阵
} MGraph;
```

(2) 算法设计

乘坐一趟直达列车相当于经过图中的一条边。因此，从 `start` 到 `end` 最少乘坐的列车趟数，就是无权图中两顶点之间的最短路径长度。可采用广度优先搜索 BFS：

1. 设置数组 `dist[]`，记录从 `start` 到各城市最少需要乘坐的列车趟数。
2. 将所有 `dist[i]` 初始化为 `-1`，表示城市顶点尚未遍历。
3. 令 `dist[start]=0`，并将 `start` 入队。
4. 每次取出队头城市 `v`，扫描邻接矩阵的第 `v` 行。
5. 对于与 `v` 相邻且尚未访问的城市 `w`，令：
 $dist[w] = dist[v] + 1$
然后将 `w` 入队。
6. BFS 首次访问 `end` 时得到的距离就是最少乘车趟数；若搜索结束仍未访问 `end`，则返回 `-1`。

(3) 算法实现

```
/*
 * 返回从 start 到 end 最少需要乘坐的列车趟数。
 * 若城市编号不合法或两城市不可达，则返回 -1。
 */
int minTrains(const MGraph *G, int start, int end)
{
    int dist[MAXV];
    int queue[MAXV]; // 用数组实现的简易队列（一次性使用）
    int front = 0, rear = 0; // 队头、队尾指针
    int i;

    // -1 同时表示城市尚未访问
    for (i = 0; i < G->numVertices; i++)
        dist[i] = -1;

    dist[start] = 0; // 起点城市，从自身到自身，距离=0
    queue[rear++] = start; // 简易队列的“入队”操作

    while (front < rear) {
        int v = queue[front++]; // 简易队列的“出队”操作

        // start == end 时也会在此正确返回 0
        if (v == end)
            return dist[v];
    }
```

```

// 扫描邻接矩阵的第 v 行，找到 v 的所有邻居
for (i = 0; i < G->numVertices; i++) {
    if (G->Edge[v][i] != 0 && dist[i] == -1) {
        // 首次入队时立即标记，避免重复入队
        dist[i] = dist[v] + 1;
        queue[rear++] = i; // 若邻居是第一次被访问，则入队
    }
}

return -1;
}

```

▼ 另一种解法（利用 2015 年 42 题结论）

由 2015 年 42 题第 (3) 小问的结论：设图的邻接矩阵为 A ，则 A^m 中第 i 行第 j 列的元素表示从顶点 i 到顶点 j 长度恰为 m 的通路条数，非零即代表至少存在一条这样的通路。

(2) 算法设计

“最少乘车趟数”= start 到 end 的最短路径边数。因此从 $m = 1$ 起逐步计算 $A^1, A^2, \dots$ ，首次出现 $A^m[start][end] \neq 0$ 时的 m 即为最少乘车趟数：

1. 若初始 $start == end$ ，无需乘车，返回 0。
2. 令 $A_m = A$ （即 A^1 ），从 $m = 1$ 递增。
3. 每轮先检查 $A_m[start][end]$ ，非零则返回当前 m 。
4. 否则令 $A_m = A_m \times A$ 得到 A^{m+1} ，继续。
5. 含 n 个顶点的图最短路径至多 $n-1$ 条边；若 m 增到 $n-1$ 仍为零，则不可达，返回 -1。

(3) 算法实现

```
#define MAXV 100

/* C = A x B, 三个矩阵均为 n x n */
void matMul(int n, int A[MAXV][MAXV],
            int B[MAXV][MAXV], int C[MAXV][MAXV])
{
    int i, j, k;
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++) {
            C[i][j] = 0;
            for (k = 0; k < n; k++)
                C[i][j] += A[i][k] * B[k][j];
        }
}

/*
 * 利用邻接矩阵幂的“通路条数”含义求最少乘车趟数。
 *  $A^m[start][end] \neq 0$  表示 start 到 end 存在长度为 m 的通路。
 */
int minTrains(MGraph *G, int start, int end)
{
    int n = G->numVertices;
    int Am[MAXV][MAXV]; // 当前的  $A^m$ ，初值为  $A^1$ 
    int tmp[MAXV][MAXV];
    int i, j, m;

    if (start == end) // 起点即终点，0 趟
        return 0;

    //  $A_m = A^1$  = 邻接矩阵
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            Am[i][j] = G->Edge[i][j];

    // 最短路径至多 n-1 条边
    for (m = 1; m <= n - 1; m++) {
```

```

    if (Am[start][end] != 0) // 首次非零, 即最少趟数
        return m;

    matMul(n, Am, G->Edge, tmp); // tmp = A^m * A = A^(m+1)
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            Am[i][j] = tmp[i][j];
}

return -1; // n-1 步内仍不可达
}

```

复杂度很恐怖：每次矩阵乘法为 $O(n^3)$ ，最多做 $n-1$ 次，故时间复杂度 $O(n^4)$ 、空间复杂度 $O(n^2)$ ，明显劣于 BFS 的 $O(n^2)$ 。此解法仅作为思维扩展，实战中仍首选 BFS。

应用题

四、(15 分) 某社交网络平台有 n 个用户，编号为 $0 \sim n-1$ 。若两个用户之间存在直接或间接的好友关系，则称其属于同一个朋友圈。初始时每个用户各自构成一个朋友圈，系统只增加好友关系，不考虑解除好友关系。

请回答下列问题：

1. 选择一种合适的数据结构，并给出其定义和初始化方法。
2. 写出查找用户所属朋友圈代表元的操作，以及合并两个朋友圈的操作。
3. 写出判断两个用户是否属于同一朋友圈的操作。

▼ 参考答案

(1) 数据结构及初始化

可采用并查集。使用双亲表示法表示若干棵树，每棵树对应一个朋友圈集合。

```
#define MAXN 1000
```

```
typedef struct {  
    int parent[MAXN];  
} DisjointSet;
```

规定：

- $\text{parent}[x] < 0$: x 为根结点， $\text{parent}[x]$ 表示集合大小；
- $\text{parent}[x] \geq 0$: $\text{parent}[x]$ 为 x 的双亲编号。

初始时每个用户单独构成一个集合：

```
void Init(DisjointSet *S, int n) {  
    for (int i = 0; i < n; i++)  
        S->parent[i] = -1;  
}
```

(2) 查找与合并操作

查找时采用路径压缩：

```
int Find(DisjointSet *S, int x) {  
    int root = x;  
    int next;  
  
    // 第一趟：沿双亲指针向上找到根结点  
    while (S->parent[root] >= 0)  
        root = S->parent[root];  
  
    // 第二趟：路径压缩，把路径上的结点直接挂到根下  
    while (x != root) {  
        next = S->parent[x];    // 先保存原来的双亲  
        S->parent[x] = root;    // 直接指向根  
        x = next;  
    }  
  
    return root;  
}
```

合并时先找到两个朋友圈的根结点，再将较小的树合并到较大的树上：

```
void Union(DisjointSet *S, int a, int b) {  
    int ra = Find(S, a);  
    int rb = Find(S, b);  
  
    if (ra == rb)
```

```

return;

/* parent 负值越小，绝对值越大，表示集合规模越大 */
if (S->parent[ra] > S->parent[rb]) {
    int temp = ra;
    ra = rb;
    rb = temp;
}

S->parent[ra] += S->parent[rb];
S->parent[rb] = ra;
}

```

采用路径压缩和按集合大小合并策略，单次合并或查询的平均时间复杂度接近 $O(1)$ 。

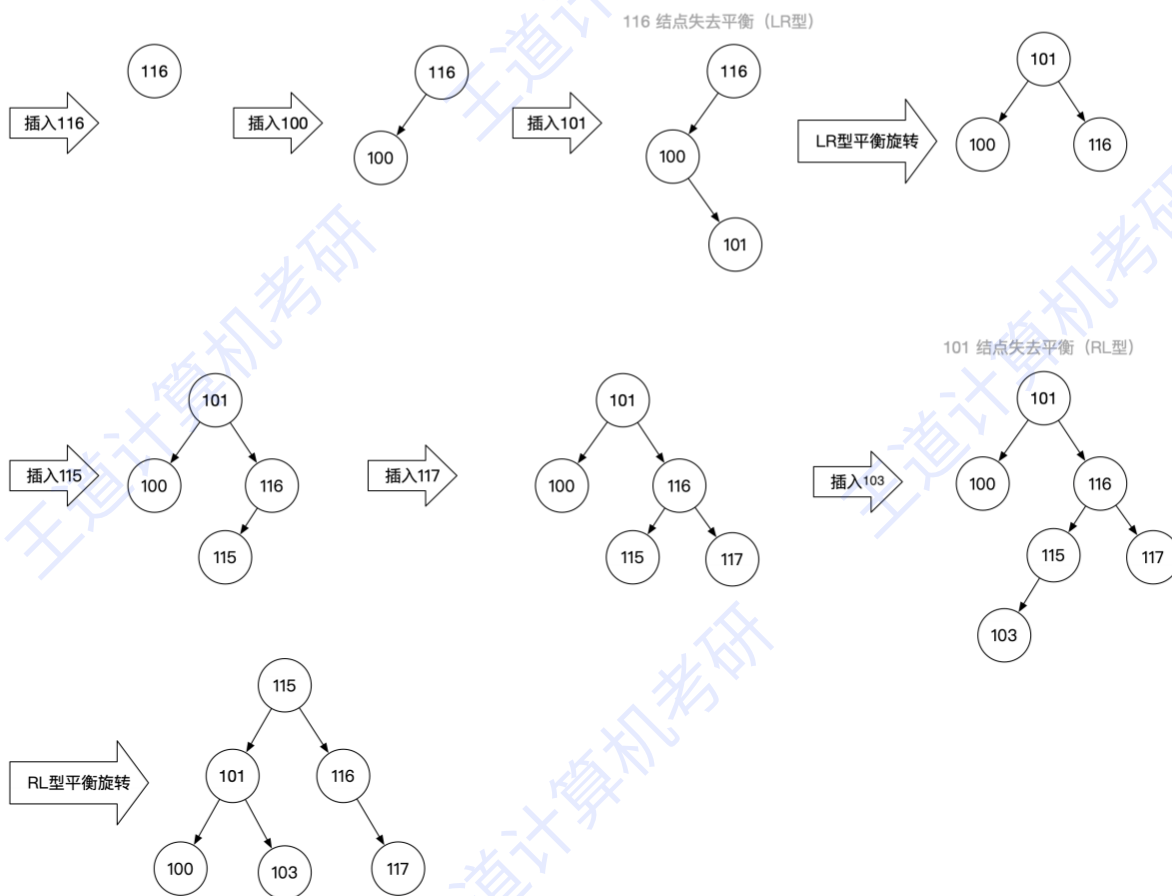
(3) 判断是否属于同一朋友圈

若两个用户的代表元相同，即 $\text{Find}(S, a) == \text{Find}(S, b)$ ；则属于同一个朋友圈。

五、(10 分) 将关键字序列 {116, 100, 101, 115, 117, 103} 依次插入到初始为空的平衡二叉树 (AVL 树)，给出每插入一个关键字后的平衡树，并说明其中可能包含的平衡调整步骤 (即：先说明是哪个结点失去平衡，然后说明做了什么平衡处理)；然后分别给出前序、中序和后序遍历该二叉树的输出结果。【中国科学院大学863 2019年】

▼ 参考答案

各关键字的插入过程如下所示：



对该二叉树的遍历结果如下

前序遍历：115，101，100，103，116，117

中序遍历：100，101，103，115，116，117

后序遍历：100，103，101，117，116，115

六、(15 分) 某软件系统包含 6 个源文件，各文件的直接依赖关系如下。若文件 **X** 依赖文件 **Y**，则必须先编译 **Y**，再编译 **X**。

文件	直接依赖的文件
A	B、C
B	D
C	D、E
D	无
E	F
F	无

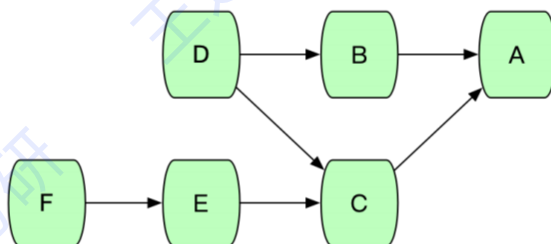
请回答下列问题：

- 以“被依赖文件指向依赖文件”的方式，画出文件依赖关系的有向图。
- 对该图进行拓扑排序。若同时存在多个入度为 0 的顶点，每次选择编号最小者，写出得到的文件编译顺序。

3. 若增加依赖关系“文件 D 依赖文件 A”，系统能否确定合法的编译顺序？简要说明理由。

▼ 参考答案

(1) 文件依赖关系的有向图



(2) 拓扑排序

每次从入度为 0 的顶点中选编号最小者输出，并删除它的全部出边（邻接顶点入度减 1）：

步骤	当前入度为 0 的顶点	选取（编号最小）	已输出序列
1	D、F	D	D
2	B、F	B	D B
3	F	F	D B F
4	E	E	D B F E
5	C	C	D B F E C
6	A	A	D B F E C A

故文件的编译顺序为：

D → B → F → E → C → A

(3) 增加“文件 D 依赖文件 A”之后

不能确定合法的编译顺序。

新增依赖相当于在图中增加一条边 $A \rightarrow D$ ，此时图中出现了回路：

- $A \rightarrow D \rightarrow B \rightarrow A$
- $A \rightarrow D \rightarrow C \rightarrow A$

图不再是**有向无环图 (DAG)**，而拓扑序列存在的充要条件是图中不含回路，因此不存在拓扑序列，系统无法确定合法的编译顺序。编译 A 前必须先编译 B，编译 B 前必须先编译 D，而编译 D 前又必须先编译 A——三者互相等待，形成**循环依赖**，谁都无法第一个开始编译。

注：拓扑排序算法本身就能自动检测回路——若排序结束时已输出的顶点数少于 n ，但图中已找不到入度为 0 的顶点，则说明图中存在回路。

另注意（易错点）：本题图中本来就有多个入度为 0 的顶点（D 和 F），说明拓扑序列**不唯一**，题目额外规定“每次选编号最小者”才使答案唯一，答题时切勿漏看这个限定条件。